

- [75] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International conference on machine learning*, pages 23965–23998. PMLR, 2022.
- [76] Hai Xiao, Jamil Tahir-Kheli, and William A Goddard III. Accurate band gaps for semiconductors from density functional theory. *The Journal of Physical Chemistry Letters*, 2(3):212–217, 2011.
- [77] Naïke Ye, Zekai Yang, and Yuchen Liu. Applications of density functional theory in covid-19 drug modeling. *Drug Discovery Today*, 27(5):1411–1419, 2022.
- [78] Duo Zhang, Xinzijian Liu, Xiangyu Zhang, Chengqian Zhang, Chun Cai, Hangrui Bi, Yiming Du, Xuejian Qin, Anyang Peng, Jiameng Huang, et al. Dpa-2: a large atomic model as a multi-task learner. *npj Computational Materials*, 10(1):293, 2024.
- [79] Alex Zunger, S.-H. Wei, L. G. Ferreira, and James E. Bernard. Special quasirandom structures. *Physical Review Letters*, 65(3):353–356, July 1990. doi: 10.1103/PhysRevLett.65.353.

Appendix Table of Contents

A	Training Details and Hyperparameters	18
A.1	Two-stage training	18
A.2	Parallelisms	18
A.3	Model	18
A.4	MPtrj and sAlex Fine-tuning	19
A.5	Training Compute Resources	19
A.6	Referencing and Normalization	20
A.7	UMA 1.2 model and training	20
B	Training Data	22
B.1	Materials	22
B.2	Molecules	22
B.3	Catalysis	22
B.4	Molecular Crystals	22
B.5	Metal-organic frameworks (MOFs)	23
C	Scaling Laws Methods	23
C.1	FLOP counting	23
C.2	Compute optimal fits	23
C.3	Fitting loss vs. parameters	24
D	Inference	24
E	Evaluations	25
E.1	UMA-1 and 1.1 Results	25
E.2	Materials	25
E.3	Catalysis	27
E.4	Molecules	27
E.5	Molecular Crystals	29
E.6	DAC	29
F	Additional MoLE Analysis	30
F.1	Expert Analysis	30
F.2	Generalization Across Architectures	30
G	Diatomics	33

A Training Details and Hyperparameters

A.1 Two-stage training

While conservative models have been found to provide reliable performance in diverse physical property prediction tasks [23], the backward pass required for force/stress prediction significantly increases inference costs. Models with direct force prediction are much more efficient, and have been found to be effective as a pre-training strategy to save compute when subsequently fine-tuned as a conservative model [7, 23]. The UMA-S and UMA-M both follow this procedure. In addition, we introduce low-precision training, max-atom batching, and max neighbor switching to further enhance the scalability and efficiency of our training process. These novel strategies are discussed in turn below. Detailed hyper-parameters are summarized in Table 9.

Precision. For pretraining, we used BF16 numerical format, commonly used in training LLMs but uncommon in MLIP models due high numerical precision requirements. In our experiments, we found that BF16 is significantly more stable than AMP-FP16 (automatic mixed precision) especially in our multi-modal setting where data distributions can vary dramatically, frequently causing gradient and loss spikes that would destabilize AMP training. However, it suffers an accuracy drop compared to AMP-FP16 and FP32. We found the degradation can be nearly completely recovered after a very small number of finetuning steps in FP32 (<1% of data).

Max-atom Batching. Due to the large differences in system topology and number of atoms/edges per system, using a fixed number of systems as batch size is infeasible. Instead we chose to use a max-atom batching scheme where we randomly pack batches that contain as close to an upper bound (max atoms) as possible without going over to guarantee an upper bound on memory usage.

Max Neighbors. For training efficiency purposes, we use a significantly smaller number of neighbors per atom during pretraining and found that it has no effect on the final performance, energy conservation, and smoothness properties of the model after finetuning with effectively “infinite” max neighbors.

A.2 Parallelisms

Although our models are designed with inference efficiency in mind, training models with a large number of MoLE experts is memory intensive. In particular, for the finetuning stages, a combination of infinite neighbors, FP32 precision, and autograd forces puts significant constraints on memory and training speed. We used a combination three parallelism training techniques summarized as follows:

- Graph parallelism (GP) [67]: Partitioning graphs across GPUs during message passing layers is used when scaling up to a large number of atoms at large model sizes. Graph partitioning is only used within a node with a fixed graph parallel rank size (2 or 4) during conservative fine-tuning stages.
- Fully-sharded data parallel (FSDP): We use the Pytorch FSDPv1 implementation on MoLE expert layers only for models with a total parameter count exceeding 1B during conservative fine-tuning when memory is scarce. Parameters are sharded within a node and replicated across nodes.
- Distributed Data Parallel (DDP): We use the standard PyTorch DDP implementation, with modifications for per-atom loss averaging and compatibility with graph parallelism.

Furthermore, we leveraged Pytorch’s Distributed Checkpointing framework to ensure saving and loading extremely large checkpoints is efficient and stable across different node configurations. Exponential moving average (EMA) is used for stable validation performance.

A.3 Model

One model for all tasks. UMA is designed for multi-task learning under diverse DFT settings. For two inputs with exactly the same atomic numbers and positions, the DFT labels will be different when different DFT settings are used. Such DFT settings include the level of theory and system total charge/spin. These task specifications are global information of the atomic system, and we process them through initial embedding layers. In this paper, five levels of theories are involved – OMat24, OC20, OMol25, OMC25, and ODAC25 all use different DFT levels of theory. OMol25 further contains systems with non-neutral charge/spin.

Table 5 Summary of main training-related hyper-parameters for the pre-training and fine-tuning stages. These hyper-parameters are shared among model sizes.

Hyper-parameter	Pre-training	Finetuning
Precision	BF16	FP32
Radius cutoff Å	6	6
Max neighbors	30	300
Force prediction	Direct	Autograd
Stress prediction	None	Autograd
Optimizer	AdamW	AdamW
Learning rate scheduling	Cosine	Cosine
Maximum learning rate	8×10^{-4}	4×10^{-4}
Warmup epochs	0.01	0.01
Warmup factor	0.2	0.2
Gradient clipping norm threshold	100	100
Model EMA decay	0.999	0.999
Weight decay	1×10^{-3}	1×10^{-3}
Energy loss coefficient	10	20
OMol energy loss coefficient	30	-
Force loss coefficient	30	2
Stress loss coefficient	-	1

Table 6 Hyper-parameters for UMA models of different sizes.

Hyper-parameters	UMA-S	UMA-M	UMA-L
Number of MoLE experts	32	32	Dense
Number of layer blocks	4	10	16
Maximum degree $L_{\max}$	2	4	6
Maximum order $M_{\max}$	2	2	2
Number of channels N_{channel}	128	128	256
Number of radial basis functions	64	128	256
Global batch size (atoms)	88k	44k	44k
Number of pre-training steps	1.68M	2.08M	2.58M
Number of fine-tuning steps	1M	545k	350k

For this iteration of the UMA models, these global information are embedded as follows:

Furthermore, to use a single model for all tasks, it is crucial to normalize the labels such that targets from different datasets fall into similar numerical ranges. We specifically design a referencing scheme that brings the diverse datasets to a similar level, detailed in Appendix B. The model hyperparameters are shown in Table 7.

A.4 MPTrj and sAlex Fine-tuning

For materials evaluations, UMA models were fine-tuned on the MPTrj [17] and sAlex [5, 62] datasets to ensure consistent DFT settings. The fine-tuning procedure is the same as eSEN-30M-OAM as documented in [23].

A.5 Training Compute Resources

The resources used to train UMA models are described in Table 8.

Table 7 Hyper-parameters for UMA models of different sizes.

Hyper-parameters	UMA-S	UMA-M	UMA-L
Number of MoLE experts	32	32	Dense
Number of layer blocks	4	10	16
Maximum degree $L_{\max}$	2	4	6
Maximum order $M_{\max}$	2	2	2
Number of channels N_{channel}	128	128	256
Number of radial basis functions	64	128	256
Global batch size (atoms)	88k	44k	44k
Number of pre-training steps	1.68M	2.08M	2.58M
Number of fine-tuning steps	1M	545k	350k

Table 8 Training Times for UMA models.

Model	Stage	GPUs in Parallel	Training Days	GPU-Type
UMA-S	Direct Pre-train	128	5	H200 140GB
UMA-S	Conserve Fine-tune	256	5	H200 140GB
UMA-M	Direct Pre-train	128	14	H200 140GB
UMA-M	Conserve Fine-tune	256	14	H200 140GB
UMA-L	Direct Pre-train	128	25	H100 80GB
UMA-L	Stress Fine-tune	128	4	H100 80GB
UMA-L	FP32 Fine-tune	128	2	H100 80GB

A.6 Referencing and Normalization

While each dataset used in this work comes with its own specific set of DFT settings, we wanted a referencing scheme that provides a way to make energy magnitudes comparable across datasets. We do this through a "heat of formation" (HOF) reference that is applied to the energies:

$$E_{ref} = E_{DFT} - \sum_i^N [E_{i,DFT} - \Delta H_{f,i}]$$

Where E_{DFT} corresponds to the total DFT energy of the system, i is the atom number, N is the total number of atoms in the system, $E_{i,DFT}$ is the DFT energy of an isolated atom i in a box, and $\Delta H_{f,i}$ is the heat of formation of atomic number i as taken directly from Mendelev [50]. $E_{i,DFT}$ was calculated using the DFT settings for each of the unique datasets in this work. Additionally, we apply a linear reference to the above energies to help with the convergence and training stability of our models. We follow the same protocol as described in the OC22 Appendix [72].

We use a custom normalization $x' = \frac{x-\mu}{\sigma}$ for all targets (energy, forces, stress), where the shift term $\mu = 0$ and the scale term $\sigma = \text{Force root mean square (RMS)}$. For the combined dataset the Force RMS is computed as the weighted average (based on the number of systems) of all individual dataset Force RMS values.

A.7 UMA 1.2 model and training

Notable changes from UMA-1.1

- Conservative fp32 training end-to-end
- Addition of charge balancing to L0 channels
- More experts 32 $\rightarrow$ 64
- Task specific output heads
- Enable single atom predictions
- Less radial basis functions 64 $\rightarrow$ 32